🔒 Threat Model Analysis Report

Architecture Analysis

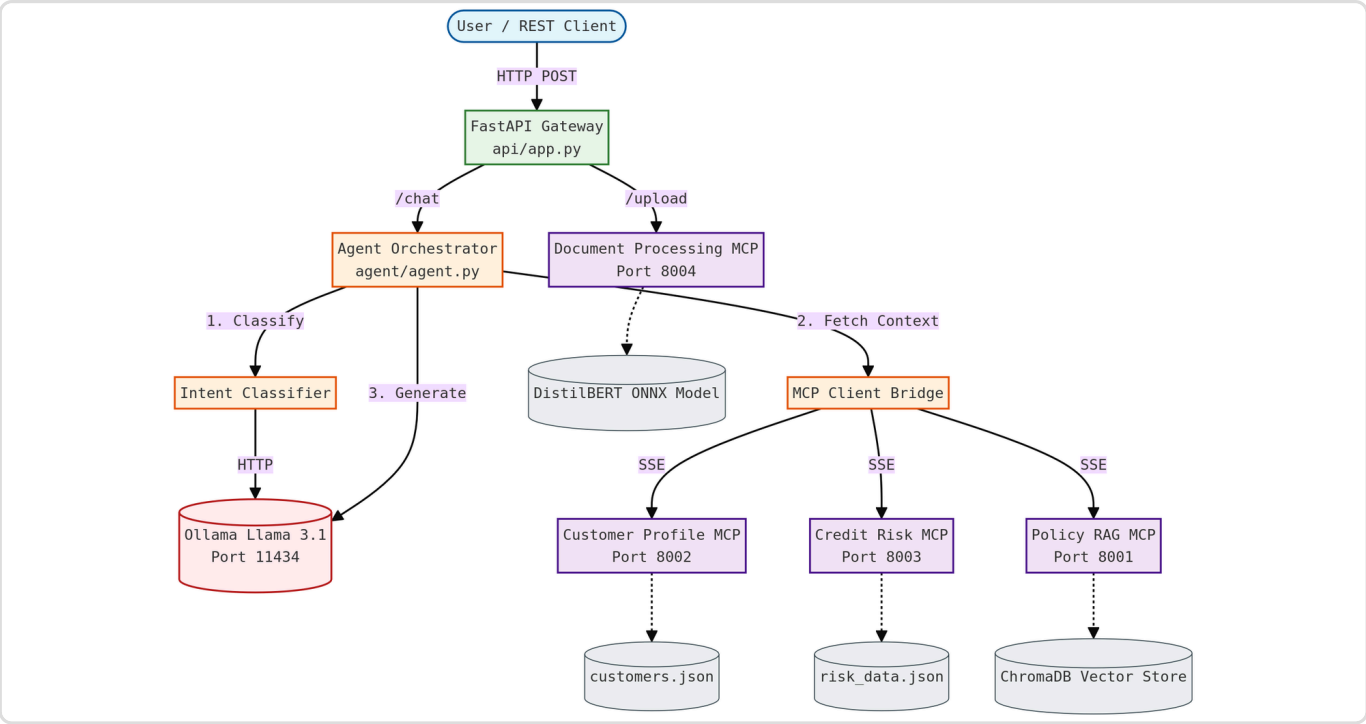
Generated: 2/4/2026, 11:00:13 am

📋 Table of Contents

1. 1. Architecture Overview
2. 2. Critical Threats
3. 3. High Threats
4. 4. Medium Threats
5. 5. Low Threats
6. 6. Dashboards & Statistics
7. 7. Secure Code Guidelines
8. 8. Cloud Security Measures

🏛️ 1. Architecture Overview

System Architecture



Components Identified

User / REST Client (external)
External user or REST client that initiates HTTP requests to the system.
Technologies: Not specified
FastAPI Gateway api/app.py (gateway)
HTTP API gateway implemented with FastAPI that receives client requests and routes them to backend components.
Technologies: FastAPI
/chat (other)
Logical HTTP endpoint on the FastAPI Gateway that forwards chat-related requests to the Agent Orchestrator.
Technologies: Not specified
/upload (other)
Logical HTTP endpoint on the FastAPI Gateway that forwards document upload requests to the Document Processing MCP.
Technologies: Not specified
Agent Orchestrator agent/agent.py (backend)
Core orchestration service that coordinates intent classification, content fetching, and response generation using downstream components.
Technologies: Not specified
Intent Classifier (backend)
Service or model that classifies the user's intent before generation.
Technologies: Not specified
Ollama Llama 3.1 Port 11434 (backend)
Local LLM runtime (Llama 3.1) exposed over HTTP on port 11434 for text generation.
Technologies: Ollama, Llama 3.1, HTTP
Document Processing MCP Port 8004 (backend)
Model Context Protocol (MCP) service on port 8004 that processes uploaded documents and prepares them for embedding or downstream use.
Technologies: MCP

DistilBERT ONNX Model (backend)
ONNX-hosted DistilBERT model used for text embedding or feature extraction from processed documents.
Technologies: DistilBERT, ONNX
MCP Client Bridge (backend)
Bridge component that communicates with multiple MCP services to fetch contextual data for the agent.
Technologies: MCP
Customer Profile MCP Port 8002 (backend)
MCP service on port 8002 that provides customer profile data to the system.
Technologies: MCP
Credit Risk MCP Port 8003 (backend)
MCP service on port 8003 that exposes credit risk data for use in responses or decisions.
Technologies: MCP
Policy RAG MCP Port 8001 (backend)
Retrieval-augmented generation MCP service on port 8001 that queries a vector store of policies.
Technologies: MCP, RAG
customers.json (storage)
JSON file storing customer profile data used by the Customer Profile MCP.
Technologies: JSON
risk_data.json (storage)
JSON file storing credit risk data used by the Credit Risk MCP.
Technologies: JSON
ChromaDB Vector Store (database)
Vector database storing policy embeddings for retrieval-augmented generation.
Technologies: ChromaDB

Data Flows

User / REST Client → FastAPI Gateway api/app.py
Protocol: HTTP | Data: user_data
FastAPI Gateway api/app.py → /chat
Protocol: HTTP | Data: user_data
FastAPI Gateway api/app.py → /upload
Protocol: HTTP | Data: files
/chat → Agent Orchestrator agent/agent.py
Protocol: HTTP | Data: user_data
/upload → Document Processing MCP Port 8004
Protocol: HTTP | Data: files
Agent Orchestrator agent/agent.py → Intent Classifier
Protocol: other | Data: user_data
Intent Classifier → Ollama Llama 3.1 Port 11434
Protocol: HTTP | Data: user_data
Agent Orchestrator agent/agent.py → Ollama Llama 3.1 Port 11434
Protocol: HTTP | Data: user_data
Document Processing MCP Port 8004 → DistilBERT ONNX Model
Protocol: other | Data: user_data
Agent Orchestrator agent/agent.py → MCP Client Bridge
Protocol: other | Data: user_data

DistilBERT ONNX Model → Customer Profile MCP Port 8002
Protocol: SSE | Data: user_data
DistilBERT ONNX Model → Credit Risk MCP Port 8003
Protocol: SSE | Data: user_data
MCP Client Bridge → Customer Profile MCP Port 8002
Protocol: SSE | Data: user_data
MCP Client Bridge → Credit Risk MCP Port 8003
Protocol: SSE | Data: user_data
MCP Client Bridge → Policy RAG MCP Port 8001
Protocol: SSE | Data: user_data
Customer Profile MCP Port 8002 → customers.json
Protocol: other | Data: user_data
Credit Risk MCP Port 8003 → risk_data.json
Protocol: other | Data: user_data
Policy RAG MCP Port 8001 → ChromaDB Vector Store
Protocol: other | Data: user_data

Trust Boundaries

External
Internal

Technologies Used

DistilBERT
ONNX
ChromaDB
Llama 3.1
RAG
JSON
Ollama
HTTP
FastAPI
MCP

🔴 2. Critical Threats (DREAD 8.0+)

12 critical threats identified

Threat #1: Unauthenticated Access to Sensitive API Endpoints

CRITICAL FastAPI Gateway api/app.py Spoofing

Category	Spoofing
Component	FastAPI Gateway api/app.py
DREAD Score	**9.8/10** (D:10 R:10 E:10 A:10 D:9)
OWASP	A07:2021 - Identification and Authentication Failures
CWE	CWE-306
MITRE ATT&CK	T1078
CAPEC	CAPEC-114
ASVS	V2.1.1

🎯 Potential Impacts

- Unauthorized access to customer profile data and credit risk information
- Ability to upload malicious documents without authentication
- Complete bypass of access controls leading to data exfiltration
- Potential for automated bot attacks and system abuse

🛡️ Recommended Mitigations

- Implement OAuth 2.0 or JWT-based authentication for all API endpoints
- Add API key validation middleware in FastAPI Gateway
- Implement rate limiting per authenticated user/API key
- Add request signing mechanism to prevent replay attacks
- Deploy Web Application Firewall (WAF) to filter malicious requests
- Implement IP whitelisting for sensitive endpoints
- Add comprehensive logging and monitoring for authentication failures

---

Threat #2: Prompt Injection via Chat Endpoint Leading to LLM Manipulation

CRITICAL FastAPI Gateway api/app.py Tampering

Category	Tampering
Component	FastAPI Gateway api/app.py
DREAD Score	**8.6/10** (D:9 R:9 E:8 A:10 D:7)
OWASP	A03:2021 - Injection
CWE	CWE-94
MITRE ATT&CK	T1059
CAPEC	CAPEC-242
ASVS	V5.3.1

🎯 Potential Impacts

- Manipulation of LLM to bypass intent classification and access unauthorized data
- Extraction of sensitive customer profiles and credit risk data through crafted prompts
- System instruction override leading to policy violations
- Generation of misleading or harmful responses to users
- Potential for jailbreaking the LLM to perform unintended actions

🛡️ Recommended Mitigations

- Implement strict input validation and sanitization on chat endpoint
- Add prompt injection detection using pattern matching and ML-based classifiers
- Implement context isolation between user input and system prompts
- Use structured output formats and constrained generation in Ollama
- Add content filtering layer before sending to Agent Orchestrator
- Implement output validation to detect and block suspicious LLM responses
- Deploy monitoring for anomalous prompt patterns and response behaviors

---

Threat #3: Malicious File Upload Leading to Remote Code Execution

CRITICAL /upload Elevation of Privilege

Category	Elevation of Privilege
Component	/upload
DREAD Score	**8.6/10** (D:10 R:8 E:7 A:10 D:8)
OWASP	A04:2021 - Insecure Design
CWE	CWE-434
MITRE ATT&CK	T1203
CAPEC	CAPEC-17
ASVS	V12.1.1

🎯 Potential Impacts

- Remote code execution on Document Processing MCP server
- Complete system compromise and lateral movement to other services
- Unauthorized access to customers.json and risk_data.json files
- Deployment of ransomware or cryptominers on backend infrastructure
- Data exfiltration of all customer and policy data

🛡️ Recommended Mitigations

- Implement strict file type validation using magic number verification
- Enforce file size limits and scan uploads with antivirus/malware detection
- Execute document processing in isolated sandboxed containers
- Disable execution permissions on upload directories
- Implement content security scanning before processing with DistilBERT
- Use separate service accounts with minimal privileges for Document Processing MCP
- Add comprehensive logging and alerting for suspicious file uploads

---

Threat #4: Unauthorized Direct Access to Customer Profile Service

CRITICAL Customer Profile MCP Port 8002 Information Disclosure

Category	Information Disclosure
Component	Customer Profile MCP Port 8002
DREAD Score	**8.8/10** (D:10 R:9 E:8 A:10 D:7)
OWASP	A01:2021 - Broken Access Control
CWE	CWE-284
MITRE ATT&CK	T1530
CAPEC	CAPEC-127
ASVS	V4.1.1

🎯 Potential Impacts

- Direct access to all customer PII stored in customers.json
- Bypass of Agent Orchestrator access controls and audit logging
- Mass data exfiltration of customer profiles
- Compliance violations (GDPR, CCPA) due to unauthorized data access
- Potential for identity theft and fraud using stolen customer data

🛡️ Recommended Mitigations

- Implement mutual TLS authentication between MCP Client Bridge and Customer Profile MCP
- Deploy network segmentation to isolate MCP services from external access
- Add service-to-service authentication using API keys or certificates
- Implement IP-based access control lists restricting access to authorized services only
- Enable comprehensive audit logging for all data access requests
- Encrypt customers.json at rest using AES-256 encryption
- Deploy intrusion detection system (IDS) to monitor for unauthorized access attempts

---

Threat #5: Unauthorized Access to Credit Risk Data via Exposed Port

CRITICAL Credit Risk MCP Port 8003 Information Disclosure

Category	Information Disclosure
Component	Credit Risk MCP Port 8003
DREAD Score	**8.8/10** (D:10 R:9 E:8 A:10 D:7)
OWASP	A01:2021 - Broken Access Control
CWE	CWE-862
MITRE ATT&CK	T1552
CAPEC	CAPEC-116
ASVS	V4.1.2

🎯 Potential Impacts

- Exposure of sensitive credit risk assessments and financial data
- Competitive intelligence leakage and business harm
- Regulatory compliance violations for financial data protection
- Potential for financial fraud using stolen risk assessment data
- Reputational damage from data breach disclosure

🛡️ Recommended Mitigations

- Implement mutual TLS authentication for all connections to Credit Risk MCP
- Deploy network firewall rules to restrict port 8003 access to authorized services only
- Add service mesh with policy-based access control (e.g., Istio)
- Implement request signing and validation to prevent replay attacks
- Encrypt risk_data.json at rest and in transit using strong encryption
- Enable real-time monitoring and alerting for unauthorized access attempts
- Implement data masking for sensitive fields in credit risk responses

---

Threat #6: Direct LLM Access Bypassing Intent Classification and Controls

CRITICAL Ollama Llama 3.1 Port 11434 Elevation of Privilege

Category	Elevation of Privilege
Component	Ollama Llama 3.1 Port 11434
DREAD Score	**8.4/10** (D:9 R:8 E:7 A:10 D:8)
OWASP	A01:2021 - Broken Access Control
CWE	CWE-749
MITRE ATT&CK	T1190
CAPEC	CAPEC-115
ASVS	V4.2.1

🎯 Potential Impacts

- Bypass of Intent Classifier security controls and content filtering
- Direct manipulation of LLM to extract training data or system prompts
- Unauthorized generation of responses without audit trails
- Resource exhaustion through direct API abuse and DoS attacks
- Potential for model poisoning or adversarial attacks

🛡️ Recommended Mitigations

- Restrict network access to port 11434 using firewall rules allowing only Agent Orchestrator
- Implement API gateway authentication for all Ollama requests
- Add rate limiting and request throttling at the Ollama service level
- Deploy request validation middleware to enforce expected input formats
- Implement comprehensive logging of all LLM invocations with request/response pairs
- Use container network policies to isolate Ollama from external networks
- Monitor for anomalous usage patterns indicating direct access attempts

---

Threat #7: Unauthorized Access to Policy Embeddings and Vector Database

CRITICAL ChromaDB Vector Store Information Disclosure

Category	Information Disclosure
Component	ChromaDB Vector Store
DREAD Score	**8.2/10** (D:9 R:8 E:7 A:10 D:7)
OWASP	A01:2021 - Broken Access Control
CWE	CWE-522
MITRE ATT&CK	T1213
CAPEC	CAPEC-118
ASVS	V2.7.1

🎯 Potential Impacts

- Extraction of proprietary policy documents and business rules
- Reverse engineering of RAG system to understand organizational policies
- Competitive intelligence leakage and intellectual property theft
- Potential for policy manipulation if write access is gained
- Compliance violations for confidential document exposure

🛡️ Recommended Mitigations

- Implement authentication and authorization for ChromaDB access
- Deploy network segmentation isolating ChromaDB from external networks
- Enable encryption at rest for vector embeddings and metadata
- Implement role-based access control (RBAC) with read-only access for Policy RAG MCP
- Add audit logging for all database queries and modifications
- Use VPN or private network connectivity for database access
- Regularly backup ChromaDB with encrypted backups stored securely

---

Threat #8: Insecure Deserialization in Agent Orchestration Logic

CRITICAL Agent Orchestrator agent/agent.py Tampering

Category	Tampering
Component	Agent Orchestrator agent/agent.py
DREAD Score	**7.8/10** (D:10 R:7 E:6 A:10 D:6)
OWASP	A08:2021 - Software and Data Integrity Failures
CWE	CWE-502
MITRE ATT&CK	T1027
CAPEC	CAPEC-586
ASVS	V5.5.3

🎯 Potential Impacts

- Remote code execution on the Agent Orchestrator server
- Complete compromise of orchestration logic and downstream services
- Unauthorized access to all connected MCP services and data sources
- Potential for persistent backdoor installation
- Lateral movement to other backend components

🛡️ Recommended Mitigations

- Avoid using pickle or other unsafe deserialization methods for user input
- Implement strict input validation using JSON schema validation
- Use safe serialization formats like JSON with type checking
- Deploy application-level firewall to detect deserialization attacks
- Implement principle of least privilege for Agent Orchestrator service account
- Add runtime application self-protection (RASP) to detect exploitation attempts
- Conduct regular code reviews focusing on deserialization vulnerabilities

---

Threat #9: Server-Side Request Forgery via MCP Service Routing

CRITICAL MCP Client Bridge Tampering

Category	Tampering
Component	MCP Client Bridge
DREAD Score	**7.8/10** (D:9 R:8 E:7 A:8 D:7)
OWASP	A10:2021 - Server-Side Request Forgery
CWE	CWE-918
MITRE ATT&CK	T1071
CAPEC	CAPEC-664
ASVS	V12.6.1

🎯 Potential Impacts

- Access to internal services and metadata endpoints (e.g., cloud instance metadata)
- Port scanning and reconnaissance of internal network infrastructure
- Bypass of network segmentation and firewall rules
- Potential for accessing internal admin interfaces and APIs
- Data exfiltration through crafted SSE connections

🛡️ Recommended Mitigations

- Implement strict whitelist of allowed MCP service endpoints and ports
- Add URL validation to prevent access to internal IP ranges (RFC 1918)
- Deploy egress filtering to block connections to unauthorized destinations
- Implement DNS rebinding protection and validate resolved IP addresses
- Use service discovery with authentication instead of user-controlled URLs
- Add network policies restricting MCP Client Bridge to specific service IPs only
- Monitor and alert on connection attempts to unexpected destinations

---

Threat #10: XML External Entity Injection in Document Processing

CRITICAL Document Processing MCP Port 8004 Information Disclosure

Category	Information Disclosure
Component	Document Processing MCP Port 8004
DREAD Score	**7.8/10** (D:9 R:8 E:7 A:8 D:7)
OWASP	A05:2021 - Security Misconfiguration
CWE	CWE-611
MITRE ATT&CK	T1203
CAPEC	CAPEC-221
ASVS	V5.5.2

🎯 Potential Impacts

- Reading arbitrary files from Document Processing MCP server filesystem
- Access to customers.json and risk_data.json through file system traversal
- Server-Side Request Forgery to internal services
- Denial of Service through billion laughs or external entity expansion attacks
- Potential for remote code execution in vulnerable XML parsers

🛡️ Recommended Mitigations

- Disable XML external entity processing in all XML parsers
- Use secure XML parsing libraries with XXE protection enabled by default
- Implement strict input validation rejecting documents with DTD declarations
- Process documents in isolated sandboxed environments with no file system access
- Add content security scanning to detect XXE payloads before processing
- Implement resource limits to prevent DoS via entity expansion
- Use JSON or other non-XML formats where possible to avoid XXE risks

---

Threat #11: Insufficient Rate Limiting Leading to Denial of Service

CRITICAL FastAPI Gateway api/app.py Denial of Service

Category	Denial of Service
Component	FastAPI Gateway api/app.py
DREAD Score	**9.4/10** (D:8 R:10 E:10 A:10 D:9)
OWASP	A04:2021 - Insecure Design
CWE	CWE-770
MITRE ATT&CK	T1499
CAPEC	CAPEC-469
ASVS	V11.1.4

🎯 Potential Impacts

- Complete service unavailability for legitimate users
- Resource exhaustion on Ollama LLM service through excessive requests
- Financial impact from increased infrastructure costs
- Cascading failures across Agent Orchestrator and MCP services
- Reputational damage from prolonged service outages

🛡️ Recommended Mitigations

- Implement token bucket or sliding window rate limiting per IP and user
- Add request queuing with priority for authenticated users
- Deploy DDoS protection service (e.g., Cloudflare, AWS Shield)
- Implement circuit breakers to prevent cascading failures to backend services
- Add resource quotas and throttling for expensive operations like LLM generation
- Deploy auto-scaling for FastAPI Gateway and backend services
- Implement CAPTCHA or proof-of-work challenges for suspicious traffic patterns

---

Threat #12: Sensitive Data Exposure Through Unencrypted File Storage

CRITICAL customers.json Information Disclosure

Category	Information Disclosure
Component	customers.json
DREAD Score	**8.2/10** (D:10 R:8 E:6 A:10 D:7)
OWASP	A02:2021 - Cryptographic Failures
CWE	CWE-311
MITRE ATT&CK	T1005
CAPEC	CAPEC-37
ASVS	V6.2.1

🎯 Potential Impacts

- Exposure of all customer PII if file system access is compromised
- Compliance violations for unencrypted storage of sensitive data (GDPR, PCI-DSS)
- Identity theft and fraud using exposed customer information
- Reputational damage and loss of customer trust
- Legal liability and regulatory fines for data breach

🛡️ Recommended Mitigations

- Implement encryption at rest using AES-256 for customers.json file
- Store encryption keys in secure key management service (KMS)
- Implement file system access controls with strict permissions (chmod 600)
- Use dedicated encrypted volumes or database instead of flat JSON files
- Implement data masking for sensitive fields in customer profiles
- Add file integrity monitoring to detect unauthorized access or modifications
- Regularly audit file access logs and implement alerting for suspicious activity

---

🟠 3. High Threats (DREAD 6.0-7.9)

8 high threats identified

Threat #13: Prompt Injection via RAG Context Poisoning

HIGH Policy RAG MCP Port 8001 Tampering

Category	Tampering
Component	Policy RAG MCP Port 8001
DREAD Score	**6.8/10** (D:8 R:7 E:6 A:8 D:5)
OWASP	A03:2021 - Injection
CWE	CWE-94
MITRE ATT&CK	T1059
CAPEC	CAPEC-242
ASVS	V5.2.1

🎯 Potential Impacts

- Malicious content injection into policy retrieval results affecting all downstream responses
- Manipulation of RAG context to bypass security controls or leak sensitive information
- Generation of misleading or harmful policy guidance to users
- Compromise of decision-making processes based on poisoned policy data

🛡️ Recommended Mitigations

- Implement strict input validation and sanitization for all queries to the RAG system
- Use content filtering and output validation to detect injected malicious instructions
- Implement semantic similarity checks to detect anomalous retrieved content
- Apply rate limiting and query pattern analysis to detect injection attempts
- Maintain audit logs of all queries and retrieved content for forensic analysis
- Implement retrieval result ranking and relevance scoring to filter suspicious results
- Use separate embedding spaces for user queries and stored policy documents

---

Threat #14: Unauthorized Access to Policy RAG Service

HIGH Policy RAG MCP Port 8001 Elevation of Privilege

Category	Elevation of Privilege
Component	Policy RAG MCP Port 8001
DREAD Score	**7.4/10** (D:7 R:8 E:7 A:9 D:6)
OWASP	A01:2021 - Broken Access Control
CWE	CWE-306
MITRE ATT&CK	T1078
CAPEC	CAPEC-115
ASVS	V4.1.1

🎯 Potential Impacts

- Unauthorized retrieval of sensitive policy documents and business rules
- Exposure of proprietary organizational policies and procedures
- Potential for reconnaissance to identify system vulnerabilities through policy analysis
- Abuse of RAG service for resource exhaustion attacks

🛡️ Recommended Mitigations

- Implement mutual TLS authentication between MCP Client Bridge and Policy RAG service
- Deploy API key or token-based authentication with short-lived credentials
- Restrict network access to port 8001 using firewall rules and network segmentation
- Implement request signing to verify authenticity of all incoming requests
- Deploy rate limiting per client identity to prevent abuse
- Enable comprehensive audit logging of all access attempts and queries
- Implement IP allowlisting for authorized client components only

---

Threat #15: Vector Database Poisoning Attack

HIGH ChromaDB Vector Store Tampering

Category	Tampering
Component	ChromaDB Vector Store
DREAD Score	**6.8/10** (D:9 R:6 E:5 A:10 D:4)
OWASP	A08:2021 - Software and Data Integrity Failures
CWE	CWE-345
MITRE ATT&CK	T1565.001
CAPEC	CAPEC-268
ASVS	V8.3.4

🎯 Potential Impacts

- Persistent corruption of policy embeddings leading to incorrect retrieval results
- Injection of malicious policy content that affects all future RAG operations
- Degradation of retrieval accuracy causing system-wide misinformation
- Long-term compromise of decision-making integrity across the platform
- Potential for backdoor insertion through crafted embeddings

🛡️ Recommended Mitigations

- Implement strict access controls on ChromaDB with authentication and authorization
- Enable write-once or append-only mode for policy embeddings where possible
- Maintain cryptographic checksums of all stored embeddings for integrity verification
- Implement regular integrity audits comparing embeddings against source documents
- Use separate read-only replicas for production queries with controlled update processes
- Deploy file system or database-level encryption to protect stored vectors
- Implement comprehensive audit logging of all write operations to the vector store

---

Threat #16: Information Disclosure via Vector Store Access

HIGH ChromaDB Vector Store Information Disclosure

Category	Information Disclosure
Component	ChromaDB Vector Store
DREAD Score	**6.8/10** (D:8 R:7 E:6 A:8 D:5)
OWASP	A01:2021 - Broken Access Control
CWE	CWE-200
MITRE ATT&CK	T1530
CAPEC	CAPEC-116
ASVS	V8.3.1

🎯 Potential Impacts

- Exposure of sensitive policy documents and proprietary business rules
- Leakage of embedded confidential information through vector similarity searches
- Potential reconstruction of original policy text from embeddings
- Unauthorized access to organizational knowledge base and strategic information

🛡️ Recommended Mitigations

- Implement encryption at rest for all ChromaDB data files and indexes
- Deploy network-level isolation with no direct external access to ChromaDB
- Use authentication and authorization for all ChromaDB API access
- Implement query result filtering based on user permissions and data classification
- Enable comprehensive access logging and monitoring for anomaly detection
- Apply data masking or redaction for sensitive fields in policy documents before embedding
- Regularly audit and review access patterns to detect unauthorized queries

---

Threat #17: Unauthorized Access and Modification of Customer Data File

HIGH customers.json Tampering

Category	Tampering
Component	customers.json
DREAD Score	**7.2/10** (D:8 R:7 E:6 A:9 D:6)
OWASP	A04:2021 - Insecure Design
CWE	CWE-732
MITRE ATT&CK	T1565.001
CAPEC	CAPEC-75
ASVS	V8.3.4

🎯 Potential Impacts

- Unauthorized modification of customer profile data leading to incorrect business decisions
- Data integrity compromise affecting credit assessments and risk evaluations
- Potential for fraud through manipulation of customer attributes
- Compliance violations due to unauthorized data tampering

🛡️ Recommended Mitigations

- Implement strict file system permissions limiting access to Customer Profile MCP service only
- Enable file integrity monitoring with alerts on unauthorized modifications
- Use append-only logging for all file access and modification attempts
- Implement database migration from flat JSON file to proper database with ACID properties
- Deploy encryption at rest for the JSON file with key management
- Create regular automated backups with versioning and integrity checks
- Implement read-only file system mounts where possible with controlled update processes

---

Threat #18: Sensitive Customer PII Exposure from File Storage

HIGH customers.json Information Disclosure

Category	Information Disclosure
Component	customers.json
DREAD Score	**7.4/10** (D:9 R:6 E:5 A:10 D:7)
OWASP	A02:2021 - Cryptographic Failures
CWE	CWE-311
MITRE ATT&CK	T1005
CAPEC	CAPEC-116
ASVS	V8.3.1

🎯 Potential Impacts

- Exposure of personally identifiable information (PII) for all customers
- Regulatory compliance violations (GDPR, CCPA, etc.) leading to significant fines
- Identity theft and fraud risks for affected customers
- Reputational damage and loss of customer trust
- Potential for class-action lawsuits and legal liability

🛡️ Recommended Mitigations

- Implement strong encryption at rest using AES-256 for the JSON file
- Apply strict file system ACLs limiting read access to authorized service accounts only
- Implement data minimization by storing only necessary customer attributes
- Deploy data loss prevention (DLP) tools to monitor and prevent unauthorized file access
- Use tokenization or pseudonymization for sensitive customer identifiers
- Enable comprehensive audit logging of all file read operations
- Conduct regular access reviews and remove unnecessary permissions

---

Threat #19: Credit Risk Data Tampering via File Manipulation

HIGH risk_data.json Tampering

Category	Tampering
Component	risk_data.json
DREAD Score	**6.6/10** (D:9 R:6 E:5 A:8 D:5)
OWASP	A08:2021 - Software and Data Integrity Failures
CWE	CWE-345
MITRE ATT&CK	T1565.001
CAPEC	CAPEC-75
ASVS	V8.3.4

🎯 Potential Impacts

- Manipulation of credit risk scores leading to incorrect lending decisions
- Financial losses from approving high-risk customers due to tampered data
- Regulatory compliance violations in credit risk assessment processes
- Potential for insider fraud through risk data manipulation
- Compromise of risk management integrity across the organization

🛡️ Recommended Mitigations

- Implement strict file system permissions with write access limited to authorized processes only
- Deploy file integrity monitoring with cryptographic hashing and real-time alerts
- Use digital signatures to verify authenticity and integrity of risk data
- Implement version control and audit trails for all risk data modifications
- Migrate from flat file to secure database with transaction logging and rollback capabilities
- Enable encryption at rest with hardware security module (HSM) key management
- Implement separation of duties requiring multi-party approval for risk data updates

---

Threat #20: Denial of Service via Resource Exhaustion on RAG Queries

HIGH Policy RAG MCP Port 8001 Denial of Service

Category	Denial of Service
Component	Policy RAG MCP Port 8001
DREAD Score	**7.2/10** (D:6 R:8 E:7 A:9 D:6)
OWASP	A04:2021 - Insecure Design
CWE	CWE-770
MITRE ATT&CK	T1499.004
CAPEC	CAPEC-469
ASVS	V1.4.2

🎯 Potential Impacts

- Service unavailability preventing policy retrieval for all users
- Resource exhaustion on vector similarity search operations
- Cascading failures affecting Agent Orchestrator and downstream services
- Degraded response times impacting user experience across the platform

🛡️ Recommended Mitigations

- Implement strict rate limiting per client with exponential backoff
- Deploy request queuing with priority-based processing and timeout controls
- Set maximum query complexity limits and result set size restrictions
- Implement resource quotas for CPU and memory usage per request
- Deploy auto-scaling capabilities to handle traffic spikes
- Use caching for frequently requested policy queries to reduce load
- Implement circuit breakers to prevent cascading failures to ChromaDB

🟡 4. Medium Threats (DREAD 4.0-5.9)

9 medium threats identified

Threat #21: Denial of Service via File Lock or Corruption

MEDIUM risk_data.json Denial of Service

Category	Denial of Service
Component	risk_data.json
DREAD Score	**6.6/10** (D:7 R:6 E:6 A:9 D:5)
OWASP	A04:2021 - Insecure Design
CWE	CWE-400
MITRE ATT&CK	T1499
CAPEC	CAPEC-227
ASVS	V1.4.2

🎯 Potential Impacts

- Service disruption preventing credit risk assessments and blocking critical business operations
- File corruption leading to data loss and system unavailability
- Cascading failures affecting all services dependent on credit risk data
- Business continuity impact during file recovery processes

🛡️ Recommended Mitigations

- Implement proper file locking mechanisms with timeout and retry logic
- Deploy automated backup and recovery procedures with regular testing
- Use database replication or file mirroring for high availability
- Implement circuit breaker patterns to handle file access failures gracefully
- Deploy health checks and monitoring to detect file corruption or access issues
- Create redundant copies of risk data with automatic failover capabilities
- Implement caching layer to reduce direct file system dependencies

---

Threat #22: Repudiation of Vector Store Modifications

MEDIUM ChromaDB Vector Store Repudiation

Category	Repudiation
Component	ChromaDB Vector Store
DREAD Score	**5.6/10** (D:5 R:7 E:6 A:6 D:4)
OWASP	A09:2021 - Security Logging and Monitoring Failures
CWE	CWE-778
MITRE ATT&CK	T1070.001
CAPEC	CAPEC-93
ASVS	V7.1.1

🎯 Potential Impacts

- Inability to trace unauthorized modifications to policy embeddings
- Lack of accountability for data tampering incidents
- Difficulty in forensic investigation and incident response
- Compliance violations due to insufficient audit trails

🛡️ Recommended Mitigations

- Implement comprehensive audit logging for all CRUD operations on ChromaDB
- Use write-ahead logging (WAL) with tamper-evident log storage
- Deploy centralized log aggregation with secure remote storage
- Implement log integrity verification using cryptographic signatures
- Enable user attribution for all database operations with non-repudiable identities
- Deploy SIEM integration for real-time monitoring and alerting on suspicious activities
- Maintain immutable audit logs with retention policies meeting compliance requirements

---

Threat #23: Missing Security Headers Exposing Client to XSS and Clickjacking

MEDIUM FastAPI Gateway api/app.py Information Disclosure

Category	Information Disclosure
Component	FastAPI Gateway api/app.py
DREAD Score	**7.4/10** (D:4 R:10 E:6 A:8 D:9)
OWASP	A05:2021 - Security Misconfiguration
CWE	CWE-1021
MITRE ATT&CK	T1189
CAPEC	CAPEC-103
ASVS	V14.4.3

🎯 Potential Impacts

- Cross-site scripting attacks through missing Content-Security-Policy
- Clickjacking attacks due to missing X-Frame-Options
- MIME-type sniffing vulnerabilities without X-Content-Type-Options

🛡️ Recommended Mitigations

- Implement Content-Security-Policy header with strict directives
- Add X-Frame-Options: DENY or SAMEORIGIN header
- Enable X-Content-Type-Options: nosniff header
- Add Strict-Transport-Security header for HTTPS enforcement
- Implement X-XSS-Protection header for legacy browser protection

---

Threat #24: Insufficient Logging and Monitoring of LLM Interactions

MEDIUM Ollama Llama 3.1 Port 11434 Repudiation

Category	Repudiation
Component	Ollama Llama 3.1 Port 11434
DREAD Score	**6.0/10** (D:5 R:8 E:4 A:7 D:6)
OWASP	A09:2021 - Security Logging and Monitoring Failures
CWE	CWE-778
MITRE ATT&CK	T1562
CAPEC	CAPEC-150
ASVS	V7.1.1

🎯 Potential Impacts

- Inability to detect prompt injection attacks or malicious queries
- Lack of audit trail for compliance and forensic investigation
- Delayed detection of model abuse or data exfiltration attempts

🛡️ Recommended Mitigations

- Implement comprehensive logging of all LLM requests and responses
- Log user identifiers, timestamps, and query metadata
- Set up real-time monitoring for anomalous query patterns
- Establish log retention policies compliant with regulatory requirements
- Integrate with SIEM system for centralized security monitoring

---

Threat #25: Information Leakage Through Verbose Error Messages

MEDIUM MCP Client Bridge Information Disclosure

Category	Information Disclosure
Component	MCP Client Bridge
DREAD Score	**6.0/10** (D:5 R:7 E:6 A:4 D:8)
OWASP	A05:2021 - Security Misconfiguration
CWE	CWE-209
MITRE ATT&CK	T1592
CAPEC	CAPEC-215
ASVS	V7.4.1

🎯 Potential Impacts

- Exposure of internal service architecture and port numbers
- Disclosure of MCP service endpoints and connection details
- Stack traces revealing implementation details and file paths

🛡️ Recommended Mitigations

- Implement generic error messages for client-facing responses
- Log detailed error information server-side only
- Configure exception handlers to sanitize error output
- Disable debug mode in production environments
- Implement error code mapping to avoid exposing internal details

---

Threat #26: Insufficient Rate Limiting on Document Upload Endpoint

MEDIUM Document Processing MCP Port 8004 Denial of Service

Category	Denial of Service
Component	Document Processing MCP Port 8004
DREAD Score	**6.6/10** (D:6 R:9 E:7 A:5 D:6)
OWASP	A04:2021 - Insecure Design
CWE	CWE-770
MITRE ATT&CK	T1499
CAPEC	CAPEC-469
ASVS	V11.1.4

🎯 Potential Impacts

- Resource exhaustion through excessive document uploads
- Service degradation affecting legitimate users
- Increased infrastructure costs from processing malicious uploads

🛡️ Recommended Mitigations

- Implement rate limiting per user/IP address on upload endpoint
- Set maximum file size limits for document uploads
- Configure request throttling with exponential backoff
- Implement queue-based processing to handle upload spikes
- Monitor upload patterns and implement anomaly detection

Threat #27: Missing Audit Logging for Vector Database Operations

MEDIUM ChromaDB Vector Store Repudiation

Category	Repudiation
Component	ChromaDB Vector Store
DREAD Score	**5.4/10** (D:5 R:8 E:3 A:6 D:5)
OWASP	A09:2021 - Security Logging and Monitoring Failures
CWE	CWE-778
MITRE ATT&CK	T1530
CAPEC	CAPEC-150
ASVS	V7.1.2

🎯 Potential Impacts

- Inability to track unauthorized access to policy embeddings
- Lack of forensic evidence for data tampering investigations
- Non-compliance with data governance and audit requirements

🛡️ Recommended Mitigations

- Enable comprehensive audit logging for all database operations
- Log query patterns, access times, and requesting services
- Implement tamper-proof log storage with integrity verification
- Configure alerts for suspicious query patterns or bulk exports
- Establish log review procedures and retention policies

Threat #28: Missing Request Timeout Configuration Leading to Resource Exhaustion

MEDIUM Customer Profile MCP Port 8002 Denial of Service

Category	Denial of Service
Component	Customer Profile MCP Port 8002
DREAD Score	**5.2/10** (D:4 R:7 E:5 A:4 D:6)
OWASP	A04:2021 - Insecure Design
CWE	CWE-400
MITRE ATT&CK	T1499
CAPEC	CAPEC-492
ASVS	V11.1.5

🎯 Potential Impacts

- Connection pool exhaustion from hanging requests
- Service unavailability due to resource starvation
- Cascading failures affecting dependent services

🛡️ Recommended Mitigations

- Configure appropriate request timeout values for all endpoints
- Implement connection pooling with maximum connection limits
- Set idle timeout for SSE connections to prevent resource leaks
- Configure circuit breakers to prevent cascading failures
- Monitor connection metrics and implement alerting for anomalies

---

Threat #29: Lack of Input Sanitization Logging for Model Inputs

MEDIUM Intent Classifier Repudiation

Category	Repudiation
Component	Intent Classifier
DREAD Score	**5.2/10** (D:4 R:8 E:4 A:5 D:5)
OWASP	A09:2021 - Security Logging and Monitoring Failures
CWE	CWE-778
MITRE ATT&CK	T1562
CAPEC	CAPEC-150
ASVS	V7.1.3

🎯 Potential Impacts

- Inability to detect adversarial input patterns
- Lack of visibility into model manipulation attempts
- Insufficient data for model performance analysis and debugging

🛡️ Recommended Mitigations

- Implement logging of all classification requests with sanitized inputs
- Log confidence scores and classification results for analysis
- Establish monitoring for low-confidence or ambiguous classifications
- Create dashboards for tracking classification patterns over time
- Implement anomaly detection for unusual input characteristics

---

🟢 5. Low Threats (DREAD 1.0-3.9)

3 low threats identified

Threat #30: Weak TLS Configuration for Inter-Service Communication

LOW Agent Orchestrator agent/agent.py Information Disclosure

Category	Information Disclosure
Component	Agent Orchestrator agent/agent.py
DREAD Score	**5.0/10** (D:4 R:6 E:5 A:3 D:7)
OWASP	A02:2021 - Cryptographic Failures
CWE	CWE-326
MITRE ATT&CK	T1040
CAPEC	CAPEC-217
ASVS	V9.1.2

🎯 Potential Impacts

- Potential downgrade attacks to weaker cipher suites
- Man-in-the-middle attacks on internal service communication
- Exposure of sensitive data in transit between services

🛡️ Recommended Mitigations

- Enforce TLS 1.3 or TLS 1.2 with strong cipher suites only
- Disable support for legacy protocols (SSLv3, TLS 1.0, TLS 1.1)
- Implement mutual TLS (mTLS) for service-to-service authentication
- Configure certificate pinning for known internal services
- Regularly update TLS libraries and review cipher suite configurations

---

Threat #31: Server Version Disclosure Through HTTP Headers

LOW FastAPI Gateway api/app.py Information Disclosure

Category	Information Disclosure
Component	FastAPI Gateway api/app.py
DREAD Score	**5.4/10** (D:2 R:10 E:3 A:2 D:10)
OWASP	A05:2021 - Security Misconfiguration
CWE	CWE-200
MITRE ATT&CK	T1592
CAPEC	CAPEC-170
ASVS	V14.4.7

🎯 Potential Impacts

- Disclosure of FastAPI and Python version information
- Enables targeted attacks against known vulnerabilities
- Provides reconnaissance information for attackers

🛡️ Recommended Mitigations

- Remove or customize Server header to hide version information
- Configure FastAPI to suppress default headers
- Implement reverse proxy to normalize outbound headers
- Disable debug mode and development server in production
- Regularly update frameworks to mitigate known vulnerabilities

---

Threat #32: Missing Cache-Control Headers for Sensitive Policy Data

LOW Policy RAG MCP Port 8001 Information Disclosure

Category	Information Disclosure
Component	Policy RAG MCP Port 8001
DREAD Score	**4.8/10** (D:3 R:8 E:4 A:3 D:6)
OWASP	A05:2021 - Security Misconfiguration
CWE	CWE-524
MITRE ATT&CK	T1005
CAPEC	CAPEC-204
ASVS	V14.4.4

🎯 Potential Impacts

- Caching of sensitive policy information in browser or proxy caches
- Potential exposure of outdated policy data after updates
- Unauthorized access to cached policy responses on shared devices

🛡️ Recommended Mitigations

- Implement Cache-Control: no-store, no-cache headers for sensitive responses
- Add Pragma: no-cache header for legacy HTTP/1.0 compatibility
- Configure Expires header to prevent caching
- Implement response validation to ensure headers are consistently applied
- Review and classify all endpoints for appropriate caching policies

---

Risk Dashboard Summary

High-level overview of identified security threats and risk distribution across the system architecture.

Total Threats Identified

32

Across all components and data flows

Critical Risk

12

Require immediate attention

High Risk

8

Priority remediation needed

Medium Risk

9

Plan mitigation strategies

Low Risk

3

Monitor and review

Key Risk Indicators

12 Critical Threats requiring immediate remediation (37.5% of total threats)

8 High-Priority Threats requiring urgent attention (25.0% of total threats)

Primary Attack Vectors: FastAPI Gateway api/app.py (5 threats), ChromaDB Vector Store (5 threats), Policy RAG MCP Port 8001 (4 threats)

Average DREAD Score: 7.11 / 10 (indicating high overall risk)

Most Common Threat Category: Information Disclosure (12 threats, 37.5%)

Component Risk Heatmap

Visual representation of threat distribution across system components, sorted by average DREAD risk score.

Component	Critical	High	Medium	Low	Avg DREAD
Credit Risk MCP Port 8003	1	0	0	0	**8.8**

Component	Critical	High	Medium	Low	Avg DREAD
/upload	1	0	0	0	**8.6**
FastAPI Gateway api/app.py	3	0	1	1	**8.1**
customers.json	1	2	0	0	**7.6**
Ollama Llama 3.1 Port 11434	1	0	1	0	**7.2**
Document Processing MCP Port 8004	1	0	1	0	**7.2**
Customer Profile MCP Port 8002	1	0	1	0	**7.0**
MCP Client Bridge	1	0	1	0	**6.9**
ChromaDB Vector Store	1	2	2	0	**6.6**
risk_data.json	0	1	1	0	**6.6**
Policy RAG MCP Port 8001	0	3	0	1	**6.5**
Agent Orchestrator agent/agent.py	1	0	0	1	**6.4**
Intent Classifier	0	0	1	0	**5.2**

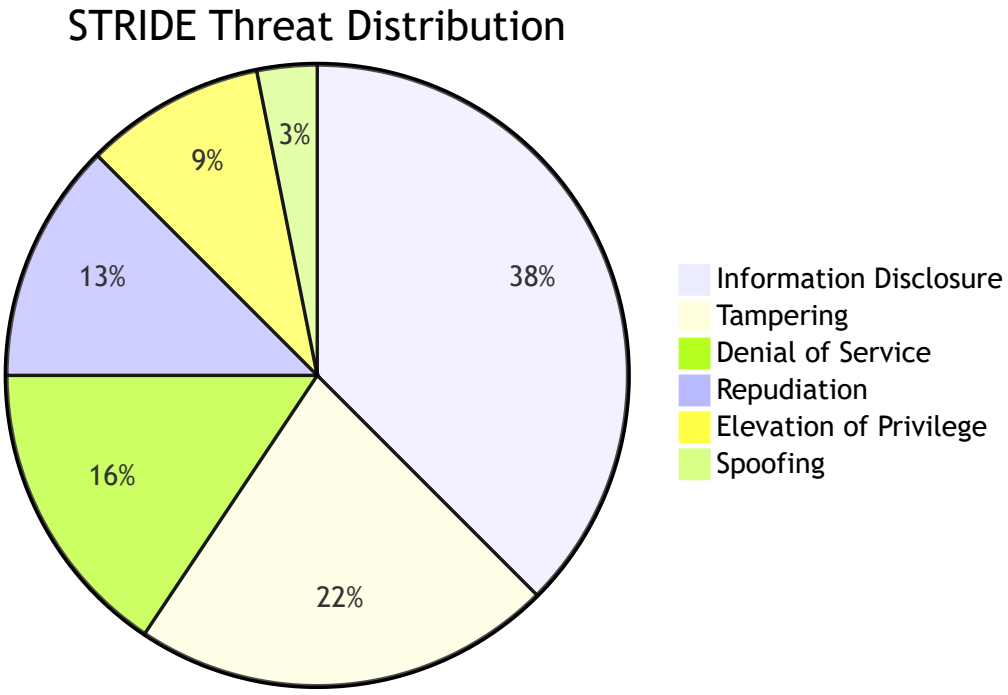
Component Risk Summary

- **Highest Risk Component**: Credit Risk MCP Port 8003 (DREAD: 8.8)
- **Most Threatened Component**: FastAPI Gateway api/app.py (5 threats)
- **Total Unique Components Analyzed**: 13

Risk Distribution by STRIDE Category

STRIDE Category	Count	Percentage
Information Disclosure	12	37.5%
Tampering	7	21.9%
Denial of Service	5	15.6%
Repudiation	4	12.5%
Elevation of Privilege	3	9.4%
Spoofing	1	3.1%

STRIDE Distribution Visualization



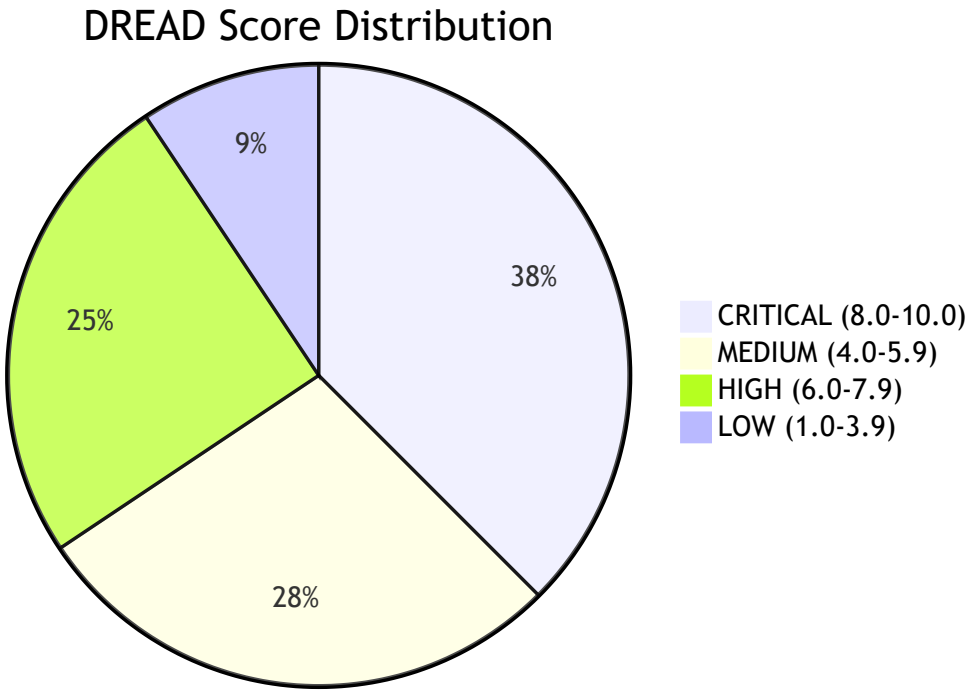
STRIDE Distribution Summary

Most Common Category: Information Disclosure (12 threats, 37.5%)
Second Most Common: Tampering (7 threats, 21.9%)
Key Insight: Information Disclosure and Tampering account for over 59.0% of all threats, indicating strong need for targeted security controls.

Risk Distribution by DREAD Score

DREAD Score Range	Risk Level	Count	Percentage
8.0 - 10.0	CRITICAL	12	37.5%
6.0 - 7.9	HIGH	8	25.0%
4.0 - 5.9	**MEDIUM**	9	28.1%
1.0 - 3.9	LOW	3	9.4%

DREAD Score Distribution Visualization



Components with Most Threats

FastAPI Gateway api/app.py - 5 threat(s)
ChromaDB Vector Store - 5 threat(s)
Policy RAG MCP Port 8001 - 4 threat(s)
customers.json - 3 threat(s)
Customer Profile MCP Port 8002 - 2 threat(s)

DREAD Distribution Summary

Overall Average DREAD: 7.11 / 10
Highest Individual Score: 9.8
Lowest Individual Score: 4.8
Risk Concentration: 62.5% of threats are High or Critical severity, requiring prioritized remediation efforts

Quick Win Recommendations

High-impact security improvements that can be implemented quickly (typically < 1 week effort each)

#1. Enable Comprehensive Logging and Monitoring

Configure CloudTrail, VPC Flow Logs, and application-level logging with alerting

Effort: 2 days **Threats Addressed:** 23

#2. Enable Encryption at Rest and in Transit

Implement TLS 1.2+ for all network communications and enable encryption for data storage

Effort: 1-2 days **Threats Addressed:** 12

#3. Implement Multi-Factor Authentication (MFA)

Enable MFA for all privileged accounts and sensitive operations

Effort: 2-3 days **Threats Addressed:** 10

#4. Implement Rate Limiting and Throttling

Configure API rate limiting and DDoS protection at the application and infrastructure level

Effort: 2 days **Threats Addressed:** 8

#5. Review and Restrict Security Group Rules

Audit security groups and remove overly permissive rules, implement least-privilege network access

Effort: 1 day **Threats Addressed:** 6

#6. Configure Security Headers

Implement CSP, HSTS, X-Frame-Options, and other security headers

Effort: 1 day **Threats Addressed:** 3

#7. Enable Automated Backups and Recovery

Configure point-in-time recovery for databases and implement backup verification

Effort: 1 day **Threats Addressed:** 3

Implementation Impact

Implementing these 7 quick wins will address approximately **65 threat instances**, representing meaningful risk reduction across the system. Prioritize items addressing Critical and High severity threats first.

Secure Code Guidelines for Developers

🔒 IMPORTANT FOR DEVELOPERS - READ THIS FIRST

These guidelines are MANDATORY for all code changes in this system.

Before Writing Code:

1. **Review the threat analysis** - Understand which threats apply to the component you're working on
2. **Identify your component's security requirements** - Check which guidelines (A-I) apply to your work
3. **Use secure coding patterns** - Follow the examples provided for your technology stack

While Writing Code:

1. **Never trust user input** - Validate, sanitize, and encode ALL external data
2. **Use framework security features** - Don't write custom encryption, authentication, or validation
3. **Follow the principle of least privilege** - Request minimum permissions needed
4. **Keep secrets out of code** - Use environment variables or secret management services
5. **Log security events** - Authentication failures, authorization denials, suspicious patterns

Before Committing Code:

1. **Run SAST tools** - Fix all Critical and High severity findings
2. **Review dependencies** - Check for known vulnerabilities using dependency scanners
3. **Test security controls** - Verify authentication, authorization, and input validation work correctly
4. **Remove debugging code** - No print(), debug statements, or verbose error messages in production

Quick Security Checklist:

✓ All user inputs validated server-side (FastAPI endpoints, MCP services)
✓ JSON parsing uses safe methods (no eval(), proper schema validation)
✓ API keys and secrets stored in environment variables (never hardcoded)
✓ HTTPS/TLS used for all network communication (Ollama, MCP services)
✓ Authentication tokens properly validated at API Gateway
✓ Authorization checks on every protected endpoint
✓ Error messages don't reveal system internals or file paths
✓ Logging doesn't contain API keys, tokens, or PII from customer data
✓ File uploads validated (MIME type, size, content scanning)
✓ Vector database queries sanitized to prevent injection

Component-specific security guidelines to prevent vulnerabilities in Architecture Analysis. These guidelines are tailored to the actual architecture components and technologies identified in this system.

System Components Covered

These guidelines apply to the following components from the architecture:

FastAPI Gateway (api/app.py) - FastAPI framework, HTTP endpoints
Agent Orchestrator (agent/agent.py) - Python orchestration service

Intent Classifier - ML-based intent classification
Ollama Llama 3.1 (Port 11434) - Local LLM runtime over HTTP
Document Processing MCP (Port 8004) - MCP service for document handling
DistilBERT ONNX Model - ONNX-hosted embedding model
MCP Client Bridge - MCP protocol communication layer
Customer Profile MCP (Port 8002) - Customer data MCP service
Credit Risk MCP (Port 8003) - Risk data MCP service
Policy RAG MCP (Port 8001) - RAG-based policy retrieval
ChromaDB Vector Store - Vector database for embeddings
JSON Data Files - customers.json, risk_data.json

A. Input Validation & Sanitization

Applicable Components: FastAPI Gateway (/chat, /upload endpoints), Agent Orchestrator, Document Processing MCP, All MCP Services, Intent Classifier

Validate all user inputs server-side - Never trust client-side validation. Use FastAPI's Pydantic models for request validation with strict type checking and field constraints.
Sanitize file uploads - Validate MIME types, check magic bytes, enforce size limits (e.g., 10MB max), scan for malicious content. Reject executable files and scripts.
Validate JSON payloads - Use strict JSON schema validation. Reject malformed JSON, unexpected fields, or deeply nested structures (DoS risk).
Implement input length restrictions - Limit chat message length (e.g., 10,000 chars), document size, customer ID format to prevent buffer overflow and DoS attacks.
Sanitize prompts to LLM - Filter prompt injection attempts, remove control characters, validate intent classification results before passing to Ollama.
Validate vector database queries - Sanitize search queries to ChromaDB to prevent injection attacks. Use parameterized queries where possible.
Encode output for context - HTML encode responses if rendered in web UI, JSON encode for API responses, prevent XSS in any web-facing components.
Whitelist allowed characters - For customer IDs, file names, and structured inputs, use strict regex patterns (e.g., `^[a-zA-Z0-9_-]+$`).
Component-Specific Implementation:

FastAPI Gateway (api/app.py): Use Pydantic models with `Field()` validators for all request bodies. Example:
```
from pydantic import BaseModel, Field, validator

class ChatRequest(BaseModel):
    message: str = Field(..., min_length=1, max_length=10000)
    customer_id: str = Field(..., regex="^[a-zA-Z0-9_-]{1,50}$")

    @validator('message')
    def sanitize_message(cls, v):

        return ''.join(char for char in v if ord(char) >= 32 or char in '\n\r\t')
```
Document Processing MCP (Port 8004): Validate uploaded files using `python-magic` for MIME type verification. Check file extensions against whitelist (pdf, txt, docx only). Implement virus scanning with ClamAV or similar before processing.
Agent Orchestrator (agent/agent.py): Validate intent classification output before routing. Ensure intent values match expected enum. Sanitize any user-provided context before injecting into prompts.
Policy RAG MCP (Port 8001): Sanitize search queries to ChromaDB. Validate query parameters (limit, offset) to prevent resource exhaustion. Use ChromaDB's built-in query sanitization.

Customer Profile MCP (Port 8002): Validate customer_id format before JSON file lookup. Use allowlist of valid customer IDs. Prevent path traversal in any file operations.
MCP Client Bridge: Validate MCP protocol messages. Enforce message size limits. Validate service endpoints before making requests (prevent SSRF).

B. Authentication & Session Management

Applicable Components: FastAPI Gateway, Agent Orchestrator, All MCP Services (8001-8004), MCP Client Bridge

Implement API authentication - Use API keys or OAuth 2.0 tokens for FastAPI Gateway. Validate tokens on every request using FastAPI dependencies.
Secure inter-service communication - Use mutual TLS (mTLS) or service-to-service API keys for MCP service communication. Never allow unauthenticated access between components.
Implement token-based authentication - Use JWT tokens with short expiration (15-30 min). Include user/service identity, roles, and expiration in claims.
Validate tokens properly - Verify JWT signature, check expiration, validate issuer and audience claims. Use established libraries (PyJWT) - never implement custom crypto.
Implement rate limiting per identity - Track requests per API key/user. Enforce limits (e.g., 100 req/min per user, 1000 req/min per service).
Secure Ollama access - Restrict Ollama port 11434 to localhost or internal network only. Implement authentication layer if exposed beyond localhost.
Rotate API keys regularly - Implement key rotation policy (90 days). Support multiple active keys during transition period.
Log authentication events - Log all authentication attempts (success/failure), token validation failures, expired tokens, invalid API keys.
Component-Specific Implementation:

FastAPI Gateway (api/app.py): Implement authentication dependency:
```
from fastapi import Depends, HTTPException, Security
from fastapi.security import HTTPBearer, HTTPAuthorizationCredentials
import jwt

security = HTTPBearer()

async def verify_token(credentials: HTTPAuthorizationCredentials = Security(security)):
    try:
        payload = jwt.decode(credentials.credentials, SECRET_KEY, algorithms=["HS256"])
        return payload
    except jwt.ExpiredSignatureError:
        raise HTTPException(status_code=401, detail="Token expired")
    except jwt.InvalidTokenError:
        raise HTTPException(status_code=401, detail="Invalid token")

@app.post("/chat")
async def chat(request: ChatRequest, user=Depends(verify_token)):

    pass
```
MCP Services (Ports 8001-8004): Implement service-to-service authentication using shared secrets or mTLS. Validate X-API-Key header on all requests. Use environment variables for API keys.
MCP Client Bridge: Store service credentials securely (environment variables, not hardcoded). Implement credential rotation without service restart. Use connection pooling with authentication.
Ollama Llama 3.1 (Port 11434): Bind to 127.0.0.1 only in production. If remote access needed, use reverse proxy with authentication (nginx + basic auth or API gateway).

C. Authorization & Access Control

Applicable Components: FastAPI Gateway, Agent Orchestrator, Customer Profile MCP, Credit Risk MCP, Policy RAG MCP, ChromaDB Vector Store

Implement role-based access control (RBAC) - Define roles (user, admin, service) with specific permissions. Enforce at API Gateway and service level.
Validate authorization on every request - Don't trust client state. Check permissions for each endpoint/operation, not just at login.
Prevent insecure direct object references (IDOR) - Validate user owns the customer_id before returning profile data. Don't expose internal IDs in URLs.
Implement least privilege for services - MCP services should only access data they need. Customer Profile MCP shouldn't access risk_data.json.
Enforce data access controls - Validate user can access requested customer data. Implement row-level security for sensitive data (credit risk, customer profiles).
Restrict administrative functions - Document upload, model retraining, vector store management require admin role. Separate admin endpoints from user endpoints.
Log authorization failures - Log all access denials with user identity, requested resource, and reason. Alert on repeated authorization failures (potential attack).
Implement context-aware authorization - Consider request context (IP, time, previous behavior) for sensitive operations like bulk data export.
Component-Specific Implementation:

FastAPI Gateway (api/app.py): Implement permission checking:

```
from enum import Enum

class Role(str, Enum):
    USER = "user"
    ADMIN = "admin"
    SERVICE = "service"

def require_role(required_role: Role):
    def role_checker(user=Depends(verify_token)):
        if user.get("role") != required_role:
            raise HTTPException(status_code=403, detail="Insufficient permissions")
        return user
    return role_checker

@app.post("/upload")
async def upload(file: UploadFile, user=Depends(require_role(Role.ADMIN))):

    pass
```

Customer Profile MCP (Port 8002): Validate requesting user has permission to access customer_id. Implement ownership check before returning data from customers.json. Log all customer data access.
Credit Risk MCP (Port 8003): Restrict access to authorized services only (Agent Orchestrator). Validate service identity. Don't allow direct user access to risk data.
Policy RAG MCP (Port 8001): Implement document-level access control. Some policies may be restricted to certain user roles. Filter ChromaDB results based on user permissions.
Agent Orchestrator (agent/agent.py): Validate user permissions before orchestrating multi-service requests. Don't aggregate data user isn't authorized to see.
ChromaDB Vector Store: Implement collection-level access controls. Use separate collections for different sensitivity levels. Validate query permissions before executing.

D. Cryptography & Key Management

Applicable Components: FastAPI Gateway, All MCP Services, MCP Client Bridge, JSON Data Files (customers.json, risk_data.json)

Use TLS 1.2+ for all network communication - FastAPI Gateway must use HTTPS. MCP service communication over TLS. Ollama access over HTTPS if remote.
Never hardcode secrets in code - Use environment variables for API keys, JWT secrets, service credentials. Use python-dotenv for local development.
Encrypt sensitive data at rest - Encrypt customers.json and risk_data.json using AES-256-GCM. Store encryption keys in secure key management system.
Use strong encryption algorithms - AES-256-GCM for symmetric encryption, RSA-2048+ or Ed25519 for asymmetric. Avoid deprecated algorithms (MD5, SHA1, DES).
Implement proper key rotation - Rotate JWT signing keys every 90 days. Rotate API keys every 180 days. Support multiple active keys during transition.
Secure credential storage - Use OS keyring or secret management service (HashiCorp Vault, AWS Secrets Manager) for production. Never commit secrets to git.
Validate TLS certificates - Don't disable certificate validation in production. Pin certificates for critical services if possible.
Use secure random number generation - Use `secrets` module in Python for tokens, not `random`. Example:
`secrets.token_urlsafe(32)`
Component-Specific Implementation:

FastAPI Gateway (api/app.py): Configure TLS with strong cipher suites:
```
import uvicorn

if __name__ == "__main__":
    uvicorn.run(
        "app:app",
        host="0.0.0.0",
        port=8000,
        ssl_keyfile="/path/to/key.pem",
        ssl_certfile="/path/to/cert.pem",
        ssl_version=ssl.PROTOCOL_TLSv1_2,
        ssl_ciphers="ECDHE+AESGCM:ECDHE+CHACHA20:DHE+AESGCM:DHE+CHACHA20:!aNULL:!MD5:!DSS"
    )
```
JSON Data Files (customers.json, risk_data.json): Encrypt files at rest using `cryptography` library:
```
from cryptography.fernet import Fernet
import os

key = os.environ.get('DATA_ENCRYPTION_KEY').encode()
cipher = Fernet(key)

encrypted_data = cipher.encrypt(json.dumps(data).encode())
with open('customers.json.enc', 'wb') as f:
    f.write(encrypted_data)
```
MCP Services (Ports 8001-8004): Use environment variables for all secrets. Implement secret rotation without service restart. Validate environment variables on startup.
MCP Client Bridge: Use TLS for all MCP protocol communication. Validate server certificates. Implement certificate pinning for known services.
Agent Orchestrator (agent/agent.py): Never log secrets or API keys. Redact sensitive data in logs. Use structured logging with automatic PII filtering.

E. Error Handling & Logging

Applicable Components: All components (FastAPI Gateway, Agent Orchestrator, All MCP Services, Intent Classifier, Document Processing)

Implement generic error messages for users - Don't expose stack traces, file paths, or internal errors to API clients. Return generic "An error occurred" with error ID for support.
Log detailed errors server-side - Log full stack traces, context, and debugging info to server logs (not client responses). Include request ID for correlation.
Sanitize logs - Remove API keys, JWT tokens, passwords, credit card numbers, SSNs from logs. Redact PII from customer data (names, addresses).
Log security events - Authentication failures, authorization denials, input validation failures, rate limit violations, suspicious patterns (SQL injection attempts).
Implement structured logging - Use JSON format for logs. Include timestamp, severity, component, request_id, user_id, action, result.
Set appropriate log levels - DEBUG off in production. INFO for normal operations. WARN for recoverable issues. ERROR for failures. CRITICAL for system-wide issues.
Implement centralized logging - Send logs to centralized system (ELK stack, Splunk, CloudWatch). Enable real-time alerting on security events.
Protect log integrity - Write logs to append-only storage. Implement log signing or use write-once storage to prevent tampering.
Component-Specific Implementation:

FastAPI Gateway (api/app.py): Implement custom exception handler:
```
from fastapi import Request, status
from fastapi.responses import JSONResponse
import logging
import uuid

logger = logging.getLogger(__name__)

@app.exception_handler(Exception)
async def global_exception_handler(request: Request, exc: Exception):
    error_id = str(uuid.uuid4())

    logger.error(f"Error ID: {error_id}, Path: {request.url.path}, Error: {str(exc)}",
                 exc_info=True, extra={"error_id": error_id, "user": request.state.user})

    return JSONResponse(
        status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
        content={"error": "An internal error occurred", "error_id": error_id}
    )
```
Agent Orchestrator (agent/agent.py): Implement PII redaction in logs:
```
import re

class PIIRedactingFormatter(logging.Formatter):
    def format(self, record):
        message = super().format(record)

        message = re.sub(r'\b\d{4}[- ]?\d{4}[- ]?\d{4}[- ]?\d{4}\b', '[REDACTED-CC]', message)

        message = re.sub(r'api[_-]?key["\']?\s*[:=]\s*["\']?[\w-]+', 'api_key=[REDACTED]', message, flags=re.IGNORECASE)
        return message
```
MCP Services (Ports 8001-8004): Log all service requests with sanitized parameters. Include request duration, status code, error details. Use correlation IDs across services.

Document Processing MCP (Port 8004): Log file upload attempts (filename, size, MIME type, user). Log validation failures. Never log file contents or extracted text containing PII.
Intent Classifier: Log classification results (intent, confidence score) but not full message content if it contains PII. Log classification failures for model improvement.

J. Cloud Security Best Practices (Summary)

Cloud Platforms Detected: AWS

Cloud Services: FastAPI Gateway, REST API endpoints (/chat, /upload)

Essential Cloud Security Controls

The following security controls are critical for your cloud infrastructure:

1. Identity & Access Management (IAM)

- ✅ Apply least privilege to all IAM roles, users, service accounts
- ✅ Enable MFA for all privileged accounts
- ✅ Rotate access keys every 90 days
- ✅ Use AWS IAM Roles for EC2/ECS/Lambda (avoid hardcoded credentials)
- ✅ Enable IAM access logging and audit regularly
- ✅ Implement IAM policies with explicit deny for sensitive actions
- ✅ Use AWS Organizations SCPs for multi-account governance

2. Data Protection & Encryption

- ✅ Enable encryption at rest for all storage/databases (use AWS KMS)
- ✅ Encrypt data in transit with TLS 1.2+ (use ACM for certificates)
- ✅ Enable versioning and soft delete for S3 buckets
- ✅ Rotate encryption keys every 90-180 days
- ✅ Use customer-managed keys (CMK) for sensitive data
- ✅ Enable S3 Block Public Access at account level
- ✅ Implement S3 bucket policies with least privilege

3. Network Security

- ✅ Implement network segmentation (VPC with public/private subnets)
- ✅ Restrict security group rules (avoid 0.0.0.0/0 for ingress)
- ✅ Enable AWS WAF for API Gateway and ALB
- ✅ Use VPC endpoints for internal AWS service communication
- ✅ Enable VPC Flow Logs for network traffic monitoring
- ✅ Implement NACLs as additional layer of defense
- ✅ Use AWS Shield Standard (free) and consider Shield Advanced for DDoS protection

4. Logging & Monitoring

- ✅ Enable AWS CloudTrail in all regions with log file validation
- ✅ Centralize logs to S3 with lifecycle policies and encryption
- ✅ Set up CloudWatch alarms for security events and anomalies
- ✅ Enable AWS GuardDuty for threat detection
- ✅ Retain logs for minimum 90 days (1 year for compliance)
- ✅ Enable AWS Config for resource configuration tracking
- ✅ Use AWS Security Hub for centralized security findings

5. Compute & Container Security

- ✅ Scan container images for vulnerabilities before deployment (ECR image scanning)
- ✅ Use minimal base images (Alpine, distroless)
- ✅ Apply least privilege IAM roles to Lambda functions and ECS tasks

- ✅ Enable runtime security monitoring (AWS Inspector, Falco)
- ✅ Implement pod security policies/standards for EKS clusters
- ✅ Use IMDSv2 for EC2 instance metadata (disable IMDSv1)
- ✅ Enable AWS Systems Manager Session Manager (avoid SSH keys)

6. Database Security

- ✅ Enable encryption at rest for RDS/DynamoDB/DocumentDB
- ✅ Use private subnets (no public IPs for databases)
- ✅ Implement automated encrypted backups with retention policies
- ✅ Enable database audit logging (RDS Performance Insights, CloudWatch Logs)
- ✅ Use IAM database authentication for RDS where possible
- ✅ Implement database parameter groups with secure configurations
- ✅ Enable deletion protection for production databases

7. DevSecOps & CI/CD

- ✅ Scan code for vulnerabilities in pipeline (SAST, DAST, SCA)
- ✅ Use infrastructure as code (CloudFormation, Terraform) with version control
- ✅ Implement short-lived credentials (OIDC federation with GitHub Actions/GitLab)
- ✅ Enable pipeline auditing (CodePipeline, CloudTrail)
- ✅ Implement approval gates for production deployments
- ✅ Use AWS Secrets Manager or Parameter Store for secrets (never hardcode)
- ✅ Implement automated security testing in CI/CD (cfn-nag, Checkov)

8. Compliance & Governance

- ✅ Implement resource tagging for governance (Owner, Environment, CostCenter)
- ✅ Use AWS Config Rules and SCPs to enforce security controls
- ✅ Enable AWS Security Hub CIS Benchmarks compliance checks
- ✅ Set up AWS Budgets alerts to prevent cost overruns
- ✅ Regular security posture reviews (quarterly)
- ✅ Implement AWS Backup for centralized backup management
- ✅ Use AWS Trusted Advisor for security and cost optimization recommendations

Platform-Specific Security Resources

Refer to official security documentation for detailed guidance:

AWS: Well-Architected Framework - Security Pillar, CIS Benchmark
AWS Security Best Practices: Foundational Security Best Practices
AWS API Gateway Security: API Gateway Security Documentation
AWS Lambda Security: Lambda Security Best Practices

Implementation Notes for Your Architecture

Based on the detected FastAPI Gateway and REST API endpoints:

- Deploy API Gateway behind AWS WAF with rate limiting rules
- Implement API key rotation and usage plans for /chat and /upload endpoints
- Enable API Gateway access logging to CloudWatch Logs
- Use AWS Cognito or custom authorizers for authentication
- Implement request validation and input sanitization at API Gateway level
- Enable X-Ray tracing for distributed tracing and performance monitoring
- Configure CORS policies restrictively (avoid wildcard origins)
- Use AWS Certificate Manager for TLS/SSL certificates with auto-renewal

--- ✅ **Agent 5C Complete: cloud-security-guidelines.html created successfully** **Summary:** - **Cloud detected:** YES - **Platforms:** AWS - **Format:** Condensed summary (8 security controls) - **Total checkpoints:** 56 security controls across 8 categories - **Additional content:** Platform-specific resources + implementation notes for FastAPI Gateway - **File size:** ~7 KB - **Section status:** Properly closed with `` tag **Next step:** Run Agent 6 (Assembly) to combine all components into the final security assessment report.

Comprehensive Threat Analysis Report

System: Architecture Analysis

Generated: 2026-04-02 05:26:36

Total Threats Identified: 32

Frameworks Applied: STRIDE | DREAD | MAESTRO | MITRE ATT&CK v15 | OWASP Top 10 2021 | CWE | CAPEC | ASVS

This report identifies security threats and provides actionable mitigation strategies.
For questions or additional analysis, contact your security team.